Laboratorium 6
Verilog – Sprzętowa realizacja dzielenia

Instytut Telekomunikacji Multimedialnej

Materiały dydaktyczne przeznaczone są WYŁĄCZNIE dla studentów Wydziału Informatyki i Telekomunikacji Politechniki Poznańskiej

Cele:

- zapoznanie z prostymi układami dzielenia.

Wstęp:

Operacja dzielenia jest kłopotliwa w realizacji wewnątrz struktury układu FPGA. Wykorzystanie operatora „/" języka Verilog powoduje wystąpienie błędu podczas próby dokonania implementacji bądź stworzenie układu dzielarki o nieznanej architekturze, wielkości i właściwościach. Aby sprzętowo zrealizować funkcję dzielenia rekomendowane jest napisanie własnego modułu.

Przebieg ćwiczenia:

1. Uruchomić program Mentor Modelsim, stworzyć nowy projekt, dodać pliki źródłowe.
2. Usunąć zawartość biblioteki „work". Przeanalizować kod:

Plik testbench.v

```
module mytestbenchmodule();

reg CLK;
initial CLK  <= 0;
always #50  CLK <= ~CLK;

reg RST;
initial
begin
    RST <= 0;
    RST <= #100 1;
    RST <= #500 0;
end

reg [31:0] i_dat_a;
reg [31:0] i_dat_b;
reg i_stb;
reg o_ack;

initial
begin
    o_ack <= 0;
    i_stb <= 0;
    #1000;
    i_dat_a <= 9;
    i_dat_b <= 2;
    i_stb <= 1;
    #100;
    i_stb <= 0;
    #4050;
    o_ack <= 1;

end

divider_simple div
(
.CLK(CLK),
.RST(RST),
```

```
.DEV_A(i_dat_a),
.DEV_B(i_dat_b),
.DEV_STB(i_stb),
.DEV_ACK(),

.RET_DIV(),
.RET_MOD(),
.RET_STB()
);

endmodule
```

Plik divider_simple.v

```
module divider_simple
(
input  wire         CLK,
input  wire         RST,

input  wire         DEV_STB,
output wire         DEV_ACK,
input  wire [31:0]  DEV_A,
input  wire [31:0]  DEV_B,

output wire          RET_STB,
output wire  [31:0]  RET_DIV,
output wire  [31:0]  RET_MOD

);

assign DEV_ACK = DEV_STB;
assign RET_STB = DEV_STB;
assign RET_DIV = DEV_A/DEV_B;
assign RET_MOD = DEV_A%DEV_B;

endmodule
```

3. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu divider_simple.
4. Przeanalizować moduł realizujący funkcję dzielenia z pomocą sukcesywnego odejmowania:

Plik divider_sub.v

```
module divider_sub
(
input  wire         CLK,
input  wire         RST,

input  wire         DEV_STB,
output wire         DEV_ACK,
input  wire [31:0]  DEV_A,
input  wire [31:0]  DEV_B,

output wire          RET_STB,
output wire  [31:0]  RET_DIV,
output wire  [31:0]  RET_MOD

);

//--------------------------------------------------------------------------------------------

assign DEV_ACK = DEV_STB && ~st2_stb;

reg [31:0] counter;
always @(posedge CLK or posedge RST) counter <= (RST) ? 0 : counter+1;

//--------------------------------------------------------------------------------------------

reg        st0_stb;
reg [31:0] st0_div;
reg [31:0] st0_mod;
reg [31:0] st0_sub;

//--------------------------------------------------------------------------------------------

reg        st1_stb;
reg [31:0] st1_div;
```

```
reg [31:0] st1_div1;
reg [31:0] st1_mod;
reg [33:0] st1_modsub; // !!!!!!!!!!!!!!!!!!
reg [31:0] st1_sub;

//-------------------------------------------------------------------------------------------

reg        st2_stb;
reg        st2_out;
reg [31:0] st2_div;
reg [31:0] st2_mod;
reg [31:0] st2_sub;

//-------------------------------------------------------------------------------------------

always @(posedge CLK or posedge RST) st0_stb <= (RST) ? 0 : st2_stb | DEV_STB;
always @(posedge CLK or posedge RST) st0_div <= (RST) ? 0 : st2_stb ? st2_div : 0;
always @(posedge CLK or posedge RST) st0_mod <= (RST) ? 0 : st2_stb ? st2_mod : DEV_A;
always @(posedge CLK or posedge RST) st0_sub <= (RST) ? 0 : st2_stb ? st2_sub : DEV_B;

//-------------------------------------------------------------------------------------------

always @(posedge CLK or posedge RST) st1_stb     <= (RST) ? 0 : st0_stb;
always @(posedge CLK or posedge RST) st1_div     <= (RST) ? 0 : st0_div;
always @(posedge CLK or posedge RST) st1_div1    <= (RST) ? 0 : (st0_div+1);
always @(posedge CLK or posedge RST) st1_mod     <= (RST) ? 0 : st0_mod;
always @(posedge CLK or posedge RST) st1_modsub  <= (RST) ? 0 : {1'b0,st0_mod}-
{1'b0,st0_sub};
always @(posedge CLK or posedge RST) st1_sub     <= (RST) ? 0 : st0_sub;
//-------------------------------------------------------------------------------------------

always @(posedge CLK or posedge RST) st2_stb <= (RST) ? 0 : st1_stb && ~ st1_modsub[33];
always @(posedge CLK or posedge RST) st2_out <= (RST) ? 0 : st1_stb &&   st1_modsub[33];
always @(posedge CLK or posedge RST) st2_div <= (RST) ? 0 : st1_modsub[33] ? st1_div :
st1_div1;
always @(posedge CLK or posedge RST) st2_mod <= (RST) ? 0 : st1_modsub[33] ? st1_mod :
st1_modsub;
always @(posedge CLK or posedge RST) st2_sub <= (RST) ? 0 : st1_sub;

//-------------------------------------------------------------------------------------------

assign RET_STB = st2_out;
assign RET_DIV = st2_div;
assign RET_MOD = st2_mod;

//-------------------------------------------------------------------------------------------
endmodule
```

5. Skorygować testbench tak, aby wykorzystywał moduł divider_sub.
6. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu divider_sub.
7. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb. Sprawdzić inne sposoby wprowadzania danych (kilka liczb w jednym testbench'u itp).
8. Uzupełnić program o wyjściowy sygnał akceptacji danych wyjściowych `input wire RET_ACK` (sugestie projektowe - nowy stan st3_ do którego wpadną dane po wykonaniu obliczeń w pętli, w st3_ obsługa wyjściowego handshake).
9. Narysować schemat blokowy modułu sukcesywnego odejmowania po modyfikacjach.
10. Porównać wyniki syntezy dwóch badanych implementacji. Uruchomić środowisko Lattice Diamond.
11. Utworzy nowy projekt (File -> New Project).
    Nadać nazwę np. „div_test”, utworzyć pusty projekt bez plików wejściowych.
    Wybrać układ który jest dostępny w laboratorium - MachXO2-4000HC i obudowę z odpowiednią ilością nóżek I/O np. CABGA256
    Rekomendowane narzędzie syntezy to Synplify Pro.
12. Dodać nowy plik projektu (Zakładka File List -> prawym na Input Files -> Add -> New File)
    Zwyczajowa nazwa głównego pliku to TOP.v
    Przekopiować zawartość potrzebnych plików źródłowych do pliku TOP (bez mytestbench!) lub dodać kolejne pliki źródłowe.
13. W zakładce Process poprosić o syntezę projektu i wygenerowanie pliku JDEC.

W takiej konfiguracji wszystkie wejściowe porty głównego modułu (powinien to być TOP.v co w zakładce File List informuje pogrubienie – jeśli tak nie jest to prawym na TOP.v -> Set Top-Level Unit) zostaną losowo przypisane do nóżek FPGA (plik .lpf zawiera informacje o dokładnych preferencjach.. dla sprawdzenia rozmiaru syntezowanego układu nie jest to bardzo istotne).

14. Wyniki syntezy można przeglądać w zakładce Output lub jeśli synteza przebiegła poprawnie w zakładce Reports (polecane pozycje to Synplify Pro oraz Map)
15. Odczytać ilość zajętych komórek logicznych (LUT4s). Która z implementacji okazała się mniejsza? Ile taktów sygnału zegarowego potrzebuje każda z dzielarek na przeprowadzenie operacji dzielenia?
16. Zmodyfikować *divider_simple.v* tak aby posiadał bufor (rejestr) danych wyjściowych, sygnał akceptacji danych wyjściowych `input wire RET_ACK` oraz bufor (rejest) danych wejściowych (*divider_simple_reg.v)*. Ponownie porównać wynik syntezy – maksymalną częstotliwość pracy układów. Ile cyklów odejmowania wykona moduł działający w oparciu o metodę sukcesywnego odejmowania w czasie jednego cyklu dzielenia układu *divider_simple_reg.v?*